

7.26 直播

本文整理学员提出的 22 个问题及回答，涵盖课程项目、AI 编程方法论、OryxOS 定位、Agent 技术选型、职业发展等。部分回答涉及需要进一步确认的项目细节，已在文中标注。

一、课程与项目相关

Q1 Dify 二次开发是最新版本吗？Human-in-the-loop 怎么实现的？

还没开始录，肯定是最新版本。workflow、flow、工作流、暂停

Dify 的 HITL 是 1.13.0 版本（2026 年 2 月）引入的 Human Input 节点。它的机制和我们训练营讲的 Loop Engineering 思想直接呼应，值得讲。

实现上：它是放在工作流画布上的一个节点，执行到这里时流程暂停，给指定的人发一个表单，人审核、填字段、点决策按钮（批准 / 拒绝 / 升级），然后流程继续。

最关键的一点是，工作流引擎会持久化暂停状态，工作流可以暂停很长时间（甚至几周）后可靠地恢复。这背后就是状态外置和可恢复执行，跟我们讲 Loop 的检查点、跟 When 的持久化恢复是同一套工程思想。

〔待确认〕课程 Dify 二开锁定的具体版本待补。如果基于的版本不低于 1.13，就直接讲这个 Human Input 节点的源码实现，正好当 HITL 教学案例；如果基于更早版本，则说明锁版本的稳定性理由，HITL 作为进阶专题单独讲。

Q2 前三个项目的扩展功能也开源共建吗？有人跟进吗？

Oryxos 我个人很感兴趣，我会长期维护

四个项目里，OryxOS 和 mq9 是真实在做的开源项目，本来就在社区共建、会持续跟进。前三个教学项目里，OryxOS 是真开源、欢迎学员贡献并且会 review；延时投递和 Dify 二开更多是教学载体，学员的扩展主要沉淀在自己的仓库作为作品集，优秀的实现可以提到 OryxOS / mq9 的生态里。

关于跟进要诚实：能保证 review 的是 OryxOS / mq9 主线的 PR，教学项目的共建更多靠学员社群互相 review。

〔待确认〕开源承诺的具体尺度（哪些真开源、亲自 review 到什么程度）请按实际情况调整，这是不能含糊的部分。

Q7 OryxOS 最初四个文档的编写思路能分享吗？

这几个项目都是我做过的，所以有明确的输入源。明确要什么，AI 知识辅助，核心是要自己输入和判断。

四个文档是一条有先后依赖的链，逐层具体，后一份建立在前一份之上：

第一份，业界调研。先搞清楚这个领域已经有什么、大家怎么解决、痛点在哪。不是为了抄，是为了找到自己的位置和差异化。没有这一步，后面的设计就是拍脑袋。

第二份，需求文档。回答要做什么、为谁解决什么问题，对应 SDD 的 specify。核心是把模糊的想法逼成明确的、边界可判断的需求。

第三份，技术方案文档。回答怎么做，对应 SDD 的 plan。技术决策、架构、模块划分在这里定，每个决策要写清为什么这么选、放弃了什么。

第四份，实施指引。回答怎么落地做出来，对应从 plan 到 tasks 的执行。

这条链的逻辑是：调研定位置，需求定目标，方案定路径，指引定执行。不擅长写的人最容易犯的错是跳步，没调研就写需求、没需求就写方案，结果每份都飘。按这个顺序一份一份往下推，就不慌了。

〔待补充〕你个人的写作习惯（先列大纲、先写反例、和 AI 怎么协作等）可以补进来，让这段更有个人方法论的味道。

Q18 Spec-Kit 主体开发 + 手动增量开发，怎么保障代码文档一致、防止腐化？

可以通过 skill 和不断的回改文档。课程主体中有很多这样的步骤。

文档腐化是所有文档驱动开发的死穴，几个机制：

一，区分活文档和死文档，只维护活文档。需求调研这类一次性、记录当时决策的文档，写完归档不用维护。真正要保持一致的是活文档：CLAUDE.md、接口契约、架构核心决策。把精力集中在少数活文档上，别想让所有文档永远同步。

二，让文档进入代码审查闭环。代码改动若涉及活文档描述的内容（比如改接口），PR 里必须同步改文档，写进 review 清单。文档和代码在同一个 PR 里一起审，就不会漂。要机制强制，不靠自觉。

三，用 CLAUDE.md 作为代码和文档的交汇点。它既是文档又直接影响代码生成，把最关键的一致性约束（接口契约、架构原则）放这里，腐化了 AI 生成立刻不对，最容易被发现，反而最容易保持鲜活。

四，接受文档一定有滞后，定期同步。不追求实时一致，设一个节奏（比如每个迭代结束）集中校对活文档和代码。承认腐化不可能零发生，用定期同步把它控制在可接受范围。

一句话：别想让所有文档永远和代码同步，识别出少数活文档，纳入 PR 审查闭环，定期校对，就够了。

〔待补充〕若 OryxOS 已用 docs-auto-sync 这类 skill 自动检测代码与文档偏差，这段可补上，更有说服力。

Q19 怎么把 Web 信息化系统改造成语音智能化？怎么保证速度和准确度？

说一下我的理解。但是需求细节不明确，不好细化。

本质是给老系统套一个自然语言 / 语音入口层，不重写老系统。三层：

输入层，语音转意图。语音先转文字（ASR），再由 LLM 把自然语言解析成结构化意图。用户说「帮我把上个月的销售报表导出来」，解析成导出报表、范围上月、类型销售。这层新加，不动老系统。

转换层，意图转 API 调用，这是核心。把老系统已有操作封装成一组明确的工具（Function Calling / MCP），LLM 把意图映射到具体的老系统 API 调用。老系统不改，只要把功能包装成工具。

执行反馈层，调用老系统既有接口执行，把结果转成自然语言 / 语音反馈。

两个担心的处理：速度上，ASR 和意图解析并行、常用意图缓存、简单操作不走大模型用规则兜底，别让每个操作都过一遍大模型。准确度上，这是最关键的，高风险操作（删除、修改、涉及钱）解析出意图后先回读给用户确认再执行，而不是直接干。这正是 HITL 人在回路，和 Dify 的 Human Input、When 的关键路径人工把关同一个思想。

一句话：不重写老系统，给它加一个语音到意图到调用已有 API 的智能入口层，高风险操作必须人工确认。这个改造模式很典型，和「改造老项目」主题契合，适合作为课程案例。

Q20 课程能不能多加实操？文档类已经很多，缺实操。

实操特别多，其实。前面的理论只是简化版。如果你想动手。

这个反馈值得重视，它戳中一个真实风险：方法论讲太多、实操太少，学员会「看懂但做不出」，正是 Q5 那位同学的困境。几个增加实操的方向：

一，每个方法论配一个「跟做」环节。讲完 Loop Engineering，就带学员真的跑一个循环（写卡片、跑命令、看它转、做 review），而不是只讲原理。

二，把文档变成边做边产出而不是先学后做。带学员针对一个真实小项目现场一份一份产出文档，在动手中学，比听讲印象深。

三，增加卡壳复盘环节。让学员自己动手撞墙再复盘，这正是最有效的学习。实操的价值不只在做出来，更在卡住再突破的过程。

一句话：「看懂但做不出」是方法论课程的最大风险，在每个方法论后面加「立刻跟做」，让学员手上一一直有活。四个真实项目本来是很强的实操载体，关键是让「做」跟紧「讲」，别攒到最后。

二、OryxOS 定位与设计

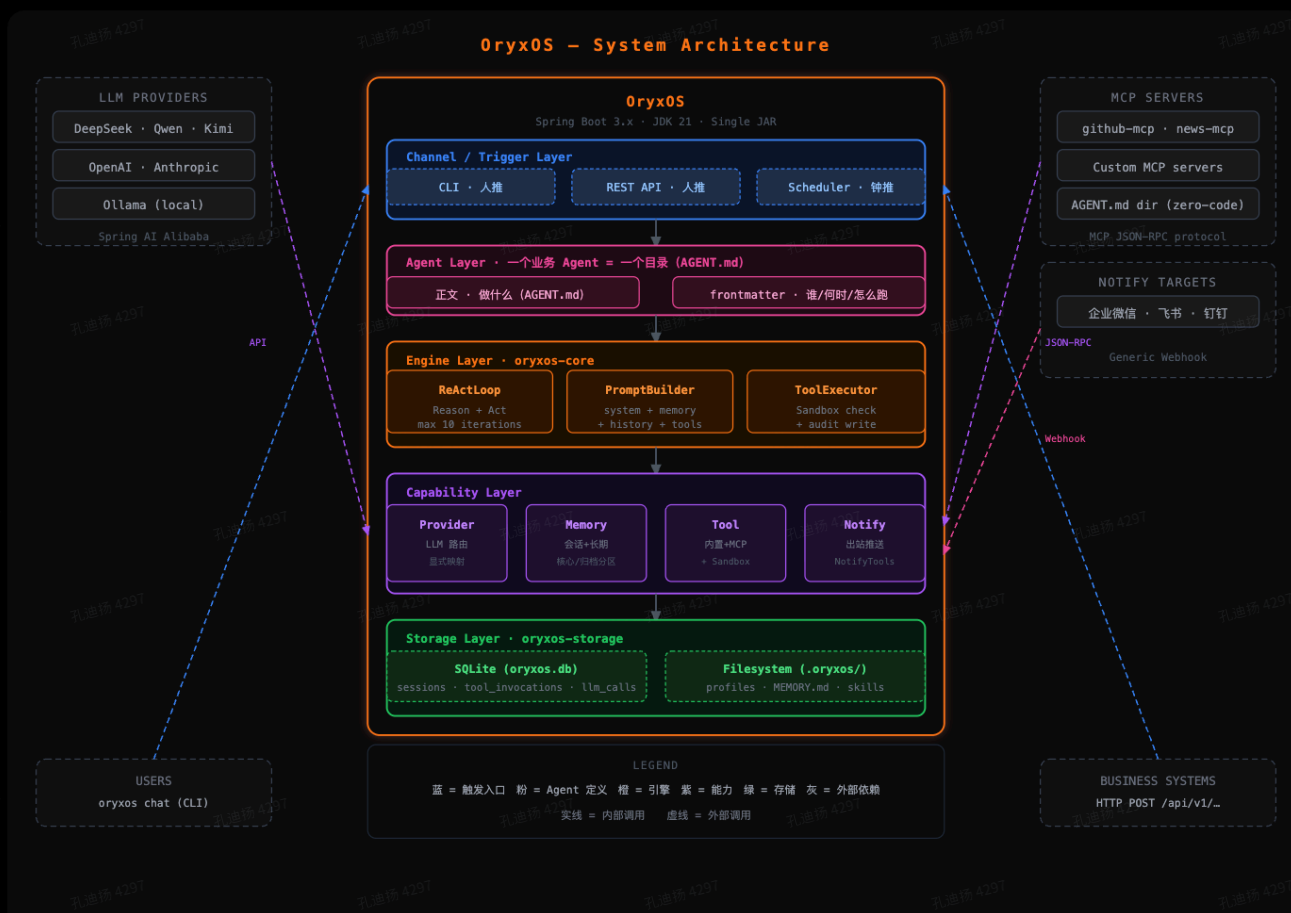
Q3 OryxOS 除了教学，企业价值是什么？存在意义只有 token 审计吗？

Hermes agent。我希望做一个专门对企业版的 hermes agent。

<https://oryx-labs.github.io/oryx/>

OryxOS 是什么

OryxOS 是企业私有部署的 Agent 操作系统：你用一句自然语言发布任务，它把任务拆解、组织一支 Agent 团队分工协作、把结果交付给你。让每一家公司，都能用自然语言跑起来自己的 Agent。



这个问题的前提有偏差。判断是「写 Agent 不难，所以底座没价值，只剩 token 审计」，其中「写 Agent 不难」成立，但「所以底座没价值」不成立。

底座的价值不在写一个 Agent，而在让一群 Agent 在企业环境里被可靠地运行、管理、协同。写一个能跑的 Agent 确实不难，但企业要的是：私有部署数据不出域、模型接入统一管理、记忆和工具作为公共能力复用、多 Agent 编排协同、故障恢复、权限与审计。这些是运行一个可靠系统的问题，不是写一个 Agent 的问题。就像写一个能跑的 Web 接口不难，但企业仍需要 K8s，因为可靠地运行、调度、治理一群服务是另一个量级的问题。

token 审计只是治理能力里最不起眼的一项，把它当成唯一价值，是把底座误解成了网关。OryxOS 对标的不是帮你写 Agent 的框架，是运行 Agent 的操作系统。而且 AI 时代写 Agent 越简单，可靠地大规模

模运行 Agent 的需求就越突出，底座的价值反而随 Agent 泛滥而上升。

一个概括：框架帮你造 Agent，底座帮你养 Agent，两件事。

Q14 OryxOS 适合做企业应用底座吗？对比 LangGraph 怎么选？有必要深入 Agent OS 吗？

当前不适合，长期在我看来肯定适合。LangGraph 是 agent 开发框架，oryxos 是 agent 运行的底座。多学总不会错。现在这种转型期，多学不会错。

这个问题其实混了两个层次不同的东西：「开发一个 Agent 应用」和「要不要用 / 深入一个 Agent OS」，是两码事。

如果目标是尽快做出一个 Agent 应用：现在就用成熟的（LangGraph 等），别碰 OryxOS，也别深入 AgentOS。你要的是造一个 Agent，成熟框架上手快、生态全、坑都被踩过了，这时 ROI 最高的选择就是站在成熟框架上。诚实说：OryxOS 现在是早期项目，成熟度和 LangGraph 不在一个量级，拿它做生产应用的底座，现在不是理性选择。

AgentOS 的价值什么时候显现？当问题从「造一个 Agent」变成「在企业里可靠地运行、治理一群 Agent」的时候：私有部署数据不出域、多 Agent 统一编排、模型 / 记忆 / 工具作为公共能力被多应用复用、统一权限审计和故障恢复。如果现在没有这类需求，那确实不需要深入 AgentOS，这个判断是对的，别为了学而学。

ROI 分场景：做单个或少量 Agent 应用，用成熟框架，不需要 AgentOS，ROI 在快速交付；企业要统一治理大量 Agent 且有私有化合规要求，才需要 AgentOS，ROI 在统一底座省掉重复建设和治理成本。

务实建议：现阶段用成熟框架做应用，把 OryxOS 当作理解 AgentOS 是什么、未来企业会需要什么的学习案例，而不是现在就上生产的底座。学它的价值是理解 Agent 泛滥后企业级治理会长成什么样，这个认知本身有价值，但和现在用什么做应用是两个决策。

Q17 agentscope / deepagents 给技术人员、AgentOS 给非技术人员，是主要区别吗？Agent 定义和 OryxOS 核心为何一致？

可以这么理解。但是我们是给开发人员用的。发展路径可以继续校准。

第一个问题，「技术 vs 非技术人员」不是主要区别，这个划分不准确。agentscope、deepagents 这类是框架 / SDK（帮开发者造 Agent），AgentOS 是运行时底座（运行和治理 Agent），区别是「造 vs 运行」，不是「给谁用」。AgentOS 同样是给技术人员用的（搭建运维底座的是工程师）。真正的区别是层次：框架帮你把一个 Agent 造出来，AgentOS 让造好的一群 Agent 在企业环境里可靠运行、统一治理。类比：框架像编程语言 / Web 框架（帮你写应用），AgentOS 像操作系统 / K8s（运行和调度应用）。

第二个问题，你的洞察是对的，点到了本质。Agent 定义（LLM / Tools / Loop / Memory / Environment）和 OryxOS 核心（LLM / Tool / ReAct / Memory / WebService）高度重合，原因正是你说的：OryxOS 本身就是为承载 Agent 而生的，所以必须把 Agent 的每个构成要素都变成自己的一等公民能力。

「更抽象了」可以说得更精确，不是抽象，是从「一次性」变成「公共服务」。在一个单独的 Agent 里，LLM / Tool / Memory 是这个 Agent 私有的、用完就完的；在 OryxOS 里，它们是底座统一提供的公共能力，被所有 Agent 共享复用，你写 Agent 不用自己接模型、不用自己实现记忆，底座给你。

所以定义一致是必然（承载 Agent 就得覆盖 Agent 所有要素），但 OryxOS 更进一层：把这些要素从每个 Agent 各写一遍的私有实现，提升为底座统一供给的公共能力。这正是操作系统的本质，操作系统提供的内存管理、文件系统、进程调度，也和一个程序需要的东西定义一致，但它把这些变成了所有程序共享的系统能力。OryxOS 之于 Agent，正是这个关系。

〔可沉淀〕这个「从私有实现到公共供给」的说法，比「运行各种 Agent 的底座」这句 slogan 更能让技术人懂，可考虑放进 OryxOS 介绍材料。

Q21 各应用出 MCP 给 OS 调用，应用内部权限审计会被突破，怎么办？

Sandbox 需要思考的事情。一点点限制。ai 安全是一个大议题。

这个担心是对的，而且很重要。当 OryxOS 统一调用各应用 MCP 时，如果用一个统一身份去调，应用就不知道到底是哪个最终用户在操作，它自己的权限审计就被绕过，这在企业级不可接受。

解法不是回退机制，而是权限穿透（身份透传）：

核心原则，OryxOS 调用应用 MCP 时必须携带最终用户身份，而不是用自己的统一身份去调，让应用依然能看到是哪个用户在操作，从而应用自己的权限体系继续生效。OryxOS 做的是传递身份，不是代替身份。

具体设计：OryxOS 作为受信任的身份传递者，不作为权限的替代者，用 OAuth 令牌透传或类似机制，把最终用户身份和权限上下文透传给被调用的应用 MCP；应用侧保留最终鉴权权，基于透传来的用户身份走自己原有的权限校验，OryxOS 不替它做鉴权决定；OryxOS 层做编排级的粗粒度管控加全链路审计，记录谁通过它发起了什么调用链，细粒度鉴权交还给应用。

关于老系统最小代价：让老系统不用改自己的权限逻辑，OryxOS 透传身份，老系统当作还是那个用户直接来调来处理，权限逻辑一行不用动，只要做「信任来自 OryxOS 的身份透传」这一个对接点。

一句话：不要让 OryxOS 用统一身份代替用户去调应用（那会绕过应用权限），要让它透传用户身份、把最终鉴权留给应用。OryxOS 管编排和审计，应用管鉴权，各司其职。这样老系统权限体系零改动，还多了一层全链路审计。

〔待设计〕身份透传、鉴权留在应用侧是业界标准方向，OryxOS 具体实现待定。这是很好的安全专题素材。

Q22 想给公司做 OryxOS，该对标现成工具的哪些功能？自己做 ROI 差还有必要吗？

这是一个需要深度思考的议题。这也是为什么现在有那么多个 agent 岗位在招聘的原因。因为大家的需求都不一样，需要根据自己对 agent 的理解去思考业务，然后做出判断。

同类不少，我们有自己的判断

单体强 Agent (Manus / Devin)

卷单个 Agent 的能力上限 我们不卷这个

可视化低代码 (Dify / Coze / n8n)

卷编排画布

我们走 md 声明式，更轻、更 GitOps

自托管个人 Agent (Hermes)

装在个人机器、越用越聪明 能力值得借鉴，但面向个人的单体

OryxOS

面向企业的分布式底座

一个底座跑一群业务 Agent · 数据留域 · 全链路审计 · 对接 MCP/A2A

OryxOS · 第一阶段总结与未来规划

7 / 24

大概率不该从零做一个 OryxOS，你的判断（ROI 差）是对的。但「别做自己的底座」不等于「没有别的思路」。

为什么对标现成工具自己做 ROI 差：像 Claude Code 的某些能力、Codex 的某些命令，是大厂投入大量资源打磨的，内部团队去复刻，效果不如、成本还高。复刻大厂的通用能力几乎注定亏。

正确思路：不做通用底座，做你公司特有的那一层。现成工具解决通用能力，直接用别自己造；你该自己做的是现成工具覆盖不了的、公司特有的部分：内部系统的接入（把老系统、内部 API 封装成工具）、公司特有的业务流程 / 权限 / 合规、把现成工具编排起来适配公司场景的薄胶水层。

所以思路转变：别做一个 OryxOS，做一个站在现成工具之上、只填补公司特有需求的薄层。通用能力全用现成的，你只做接入内部系统加适配内部流程加统一治理这一层，ROI 就正了。

什么时候才值得自研重底座：公司有大量 Agent 要统一治理、且有强私有化合规要求、现成工具满足不了时。没到这个规模，薄层就够。

一句话：别对标现成工具去复刻（那必亏），把现成工具当能力层直接用，你只做接入公司内部系统加适配内部流程这一薄层，那才是别人替代不了、ROI 为正的部分。判断要不要自研底座的标准是：你有没有现成工具满足不了的、公司特有的治理需求。

三、Agent 技术与选型

Q10 ReAct loop 和 embabel 的目标驱动 OODA loop 怎么看？embabel 值得投入吗？

两条路径而已，我不做选择题都学。react 还是主流，ooda 咋说呢，可以看，不过我短期觉得投入的重要性没 react 强。embabel 是向量？

先说本质区别。ReAct 是让 LLM 每一步自己想下一步做什么（推理和行动交替，下一步由模型即时决定）。embabel 的 GOAP（目标导向行动规划）借用游戏 AI 算法，你描述想达成的目标和可用的动作，一个非 LLM 的规划算法算出从当前状态到目标的路径，每个动作后重新规划，形成 OODA 循环。

关键差异就一点：谁来决定下一步。ReAct 是 LLM 决定，embabel 是确定性算法决定。因为规划器是确定性的、不是让语言模型决定下一步，步骤序列可解释，可以在执行前检查整个计划。ReAct 灵活但不可预测、可能跑偏、每步都烧 LLM 调用；GOAP 可解释、可预测、减少不必要的 LLM 调用，但要求你能把问题建模成明确的目标、动作、前置后置条件。

判断分三层：

一，技术方向上，embabel 代表的「把规划从 LLM 手里拿出来交给确定性算法」是正确且重要的思路，尤其在企业级、要可靠可解释的场景。它和 Loop Engineering 的内核一致，都是不完全信任 LLM 自主决策，用外部结构约束它。能注意到这个区别，说明有品味。

二，值不值得投入看场景。embabel 是 Spring 创始人 Rod Johnson 2025 年做的 JVM 原生 Agent 框架，构建在 Spring AI 之上。如果你是 Java / Spring 技术栈、要做企业级可解释的 Agent，非常值得关注（背景硬、方向对、和 Java 生态贴合）；如果是别的栈、或做探索性强难以预先建模成 GOAP 的任务，ReAct 类仍更合适。不是二选一，是看任务性质，目标动作清晰的用 GOAP，开放探索的用 ReAct。

三，给学员的务实建议：先把 ReAct 这个主流范式吃透（90% 的场景和岗位还是它），再把 embabel / GOAP 作为进阶去理解「还有另一种不依赖 LLM 决策的思路」。理解两者的分界（谁决定下一步）比押注某个框架更重要。框架会变，这个判断分界不会变。

〔可加〕OryxOS 自研 ReAct 不套框架，末尾可加一句 OryxOS 的选择及理由，把它变成一个和项目相关的活案例。

Q11 Rust 技术栈有成熟的、能用于企业级数据方向的智能体框架吗？

有，2026 年 Rust 智能体生态已从实验走向生产可用。

最成熟的是 Rig（0xPlaygrounds 出品，MIT 许可），到 2026 年 6 月约 7600 星，20+ 模型 provider 集成、OpenTelemetry 可观测、MCP 支持、WASM 兼容，有 Neon、St Jude、Nethermind 等生产用户。注意定位：Rig 严格说是库不是框架，给你搭 LLM 应用的构件（客户端、函数 schema、响应解析），不是完整的多 Agent 编排框架。企业级数据方向，Rig 作为底层 LLM 集成层最稳。

其他按需选：AutoAgents（用 Ractor actor 模型做多 Agent 编排）、rs-graph-llm（图式多 Agent 工作流，有物流场景 99.99% 可用性的生产报告）、OpenFANG（号称完整的 Agent OS）。

一个技术洞察：这些 Rust Agent 框架几乎全部建在 Tokio 之上，Tokio 的调度、mpsc、JoinSet、select! 已成 Rust 异步 Agent 开发的事实标准。这和 RobustMQ 技术栈同源，Rust + Tokio 的分布式功底迁移到 Agent 基础设施方向几乎零成本。

选型建议：企业级数据 Agent，用 Rig 做 LLM 集成层加自己按业务搭编排，比套一个大而全的框架更可控，也符合核心机制自己掌握的判断。

Q15 企业开发 Agent OS，更多是自研核心机制还是用 Agent SDK？

基本都想省力，从开源二开。这也是大家为什

现状是分层的：

应用层的公司倾向用 SDK / 框架。大多数做 Agent 应用的公司不会自己实现 Loop、Context、Memory，用现成的，把精力放在业务上。这是理性的，这些机制自己造轮子 ROI 低。

做平台 / 基础设施的公司倾向自研核心机制。当要做的是给整个组织提供 Agent 底座、或有强私有化可控要求时，核心机制（尤其 Loop、Memory、多 Agent 编排）会倾向自研，因为这些是核心竞争力和差异化，握在自己手里才可控。这也是 OryxOS 自研 ReAct 不套框架的理由。

分界线：这个机制是不是你的核心差异化。是（你就是做 Agent 平台的）就自研核心、掌握主动权；不是（你做业务应用）就用 SDK，别重造轮子。

务实结论：绝大多数公司应该用 SDK，只有少数做底座 / 平台、或有强可控需求的才自研核心机制。看到大厂自研就跟着自研是常见误区，大厂自研是因为它在做平台，你如果在做应用，用 SDK 才对。

四、职业发展与能力提升

Q5 看课件能看懂但讲不出来，怎么成长？

动手，动起手来，你就成长了。花时间就行了

「看懂」和「讲得出」之间隔着的，正是真正的理解。看懂是被动识别（这个我认识），讲出是主动重构（我能从零把它组织出来），后者才是面试和工作要的能力。这个 gap 不是靠多看几遍课件能填的，多看只会强化「看懂」的错觉。

三个动作：一，费曼式复述，合上课件假装给新人讲这个知识点，讲不下去的地方就是真正的盲区，回头针对性补。二，动手复现，课件里的每个设计不看答案实现一遍，卡住的地方就是理解没到位的地方，这正是刻意保留卡壳。三，用输出倒逼输入，写博客、做分享、回答别人的问题，逼自己把看懂重新组织成讲清。

一句话：理解不是输入出来的，是输出出来的。现在讲不出不是没学会，是还没到重构这一步。不是天赋问题，是练习方式的问题。

Q12 判断力太宽泛、没有可衡量标准和训练手册，怎么提升？

动手，看你能不能写出来。能不能做出东西。没诀窍，也很简单粗暴。

先破执念：判断力本来就没有、也不该有一本标准教程，因为它不是知识，是判断。知识能出教程，判断只能靠大量案例加反馈练出来。但没标准手册不等于没法练。

先把宽泛的判断力收敛成可练的核心：判断力等于在信息不全时，做出且能说清理由的取舍。衡量标准就一条：你能不能讲清为什么选 A 不选 B、放弃了什么。讲不清就是没到，讲得清且经得起追问就是有了。这和面试那套三层追问是同一个尺子。

提升路径从窄到宽三步：一，从技术选型这个最容易练的切口入手，每次做技术决策强制写三行，选了什么、放弃了什么、为什么，写多了判断从感觉变成有依据。二，用复盘补反馈，工作里反馈慢，

回头看三个月前的决策，对了为什么、错了错在哪，把慢反馈变成主动复盘。三，大量读别人的判断，看优秀项目的设计文档、技术方案里的 trade-off 讨论、资深人的复盘，判断力很大程度是见过足够多的案例。

一句话：别找教程了，判断力没有教程。找一个小切口（技术选型），开始写为什么，然后复盘。判断力是练出来的，不是学出来的。这也是课程为什么用真实项目而不是讲理论，判断力只能在真实取舍里长。

Q13 公司有严格部署运维体系，只有 demo 才能设计 workflow，实际工作怎么用、怎么提升？

业界所有公司都在 AI 化开发运维体系。即使没动手，也在想。当有一天你们公司的需要 AI 化。有环境就用，没环境就学。

一个误解要纠正：workflow 设计力不等于从零搭一套需求到运维的全流程。在成熟公司里全流程是现成的，轮不到你设计，但这不代表 workflow 设计力没用武之地，它只是换了个尺度。

真正的工作流设计力在日常里体现在你自己那一段的微观 workflow 上。比如接到一个需求，怎么组织理解需求、拆任务、用 AI 实现、自测、提 PR 这个属于你个人的循环？这就是 workflow 设计。公司给了大流程骨架，但你在自己这一格里怎么高效流转，完全是你能设计、能优化的。Loop Engineering、任务卡片、grill 需求全是这个尺度的 workflow 设计力，在严格的公司体系里照样能用，不冲突任何部署运维规范。

提升做法：把你重复做的事固化成个人 workflow。你每天都在读需求、写代码、调试、提交，观察哪一步最耗时、最容易出错，把它模板化、自动化（一个 checklist、一个 skill、一个脚本）。这就是在练 workflow 设计力，而且立刻见效。

一句话：大公司不让你设计大流程，但你永远能设计你自己那一段的 workflow。workflow 设计力练的是后者，它在任何体系里都有用武之地。

Q16 未来程序员价值就是写四个文档？一般人写不好交给架构师就行，岗位会大减？核心竞争力是写高质量文档？

多写。这点可以多聊

这个推论有个关键跳跃是错的：把程序员的价值等于写文档。文档不是价值本身，是判断力的载体。一份高质量的技术方案之所以有价值，不是因为文档这个形式，是因为它背后是想清楚了架构、权衡了选型、预见了风险的判断力。让一般人写和让架构师写，产出天差地别，差的不是文笔，是文档背后那个人的判断深度。所以核心竞争力是判断力，文档只是它外化出来的形式。

岗位会不会大减？会变，但不是简单减少，是结构性转移。纯执行的岗位（把明确需求翻译成代码）确实会大幅减少，因为这部分 AI 接管了；但需要判断的岗位（决定做什么、怎么架构、怎么权衡、对结果负责）不会减少，反而更值钱，因为 AI 把执行变便宜后，判断成了稀缺资源。这正是课程和直播的核心论点：执行交给 AI，判断留给人。

至于一般人写不好交给架构师，这恰恰说明了方向：未来每个程序员都要往能做判断的方向走，而不是停在能执行。不是岗位没了，是只会执行的人没竞争力了，能判断的人（不管叫不叫架构师）更值

钱了。

一句话：别担心未来是不是只需要写文档，要担心你的判断力够不够撑起一份好文档。岗位不会消失，会向判断力集中。你要练的不是文笔，是判断。

Q9 没维护过开源项目，新手想参与，要提前学什么？

动手，在我们这个项目练手。你交钱了，我需要给你交付这部分。关键是你想动手。

门槛主要在协作规范而非技术。技术之外先掌握协作机制：

一，Git 协作流，fork、branch、PR、rebase 解冲突，这是参与开源的基本功，比想象中重要。二，读懂项目规矩，CONTRIBUTING.md、issue 模板、PR 模板、代码规范，每个项目都有自己的规矩，先读再动手。三，从小处入手，第一个 PR 别挑大功能，从修文档错别字、补测试、修一个 good-first-issue 开始，先把提 PR 到被合并这个完整流程走通一遍，建立信心。四，沟通礼仪，issue 里把问题讲清楚（复现步骤、环境、期望），PR 描述写清楚改了什么为什么，尊重维护者的时间。

技术上：能读懂项目架构文档、能在本地把项目跑起来、能定位到你该改的那块代码。

心态建议：开源新手最大的障碍是怕自己代码烂被拒，但维护者其实欢迎任何认真的贡献，哪怕是小的。从 good-first-issue 开始走通一次就入门了。OryxOS 和 mq9 都欢迎从这种小 PR 开始。

Q4 用 git submodule 管理前后端仓库，能细讲吗？

这个我认为不是重点吧。

先给一个诚实判断：git submodule 是能解决问题、但公认体验割裂的方案，用它之前先确认你真的需要它。

适用场景：前后端是两个独立仓库、独立发布、可能被多个项目复用，但你想在一个总装仓库里锁定它们的特定版本一起部署。价值是版本锁定（父仓库记录子仓库的确切 commit）和仓库解耦。代价是操作繁琐，clone 要 recursive、拉代码要 submodule update、子模块改动要分两层提交、团队里总有人忘了更新导致版本对不上。

对训练营项目的建议：如果前后端就是一起开发、一起发布、不被别人复用，用 monorepo（一个仓库放前后端两个目录）比 submodule 省心得多。submodule 更适合我要引用一个独立演进的外部仓库的场景，比如把 mq9 作为子模块引入某个项目。为用而用 submodule 是常见的过度工程。

〔可加深〕如需更实用，可补一个 submodule / monorepo / subtree 三选一决策表。

Q6 老师用什么模型？有稳定的中转平台推荐吗？

我个人没有，问群的同学

〔待填写〕主力模型请自行填写。中转平台方面，可以问助教老师。

补充一个原则（呼应「工具会来来去去」的立场）：模型优先用官方渠道，中转站尽量少碰，碰也选口碑稳的，别为省小钱担数据和跑路风险。

Q8 Spec-Kit / OpenSpec 产出的文档每个都要 review 吗？重点看哪里？

懂了就不看，不懂就好好看。sdd 的标准化就可以。

不是每个都要同等力度 review，要分轻重。核心判断：review 的力度应该和这份文档的下游影响成正比。

重点 review（逐字看）：spec（需求规格）和 plan（技术方案）。它们是源头，错了会污染后面所有任务，spec 里一个模糊需求会让后面十个任务全跑偏。这两份重点看：需求边界清不清、验收标准可不可判定、技术选型理由成不成立、有没有漏掉的约束。

快速过（扫一眼确认没跑偏）：tasks（任务拆分）。它是 spec 和 plan 的机械展开，只要上游对了，任务清单一般不会错得离谱，扫一遍看粒度合不合适、有没有遗漏就行。

一句话：越上游的文档越要重 review，越下游的越可以快速过。把精力压在 spec 和 plan 上，这是投入产出比最高的地方。这也和 Loop 里改卡片比跟 AI 掰扯有效是同一个道理，修上游比修下游高效。

我的问题

- 为什么想买训练营，动力是什么
- 你们会想从训练营得到什么
-